

Benchmarking Local Vision-Language Models on an Edge Device for the Visually Impaired

Fatima Sarwar

Dept. of Mechatronics Engineering
College of E&ME, NUST
Islamabad, Pakistan

fsarwar.mts44ceme@student.nust.edu.pk

Baseer Ayaz Raja

Dept. of Mechatronics Engineering
College of E&ME, NUST
Islamabad, Pakistan

bayaz.mts44ceme@student.nust.edu.pk

Muhammad Ibrahim

Dept. of Mechatronics Engineering
College of E&ME, NUST
Islamabad, Pakistan

mibrahim.mts46ceme@student.nust.edu.pk

Ayesha Zeb

Dept. of Mechatronics Engineering
College of E&ME, NUST
Islamabad, Pakistan
ayesha.zeb@ceme.nust.edu.pk

Umar Shahbaz Khan

Dept. of Mechatronics Engineering
College of E&ME, NUST
Islamabad, Pakistan
u.shahbaz@ceme.nust.edu.pk

Usman Asad

Dept. of Mechatronics Engineering
College of E&ME, NUST
Islamabad, Pakistan
usman.asad@ceme.nust.edu.pk

Abstract—Visually Impaired People (VIP) require reliable scene understanding for safe navigation in complex environments. While state-of-the-art Vision-Language Models (VLMs) can describe scenes in great detail, they typically rely on cloud servers, raising issues such as privacy risks and continuous internet dependence. This paper explores a local solution by benchmarking five open-source VLMs running entirely on NVIDIA’s Jetson Orin Nano (8 GB) via the Ollama (llama.cpp) framework. The models were tested on the VizWiz-VQA dataset, which consists of real images captured by blind users. The models’ performance was evaluated on hardware metrics such as latency, throughput, power consumption, etc., as well as semantic reliability using CLIPScore and POPE benchmark to detect hallucinations. The results demonstrate that while the larger models (gemma3:12b & minstral-3:8b) fall short in hardware benchmarks, they are significantly less likely to provide hallucinatory responses.

Index Terms—Image captioning, local vision-language models (VLM), edge devices, visually impaired people (VIP)

I. INTRODUCTION

Visual Impairment affects 2.2 billion people worldwide [1]. Visually Impaired People (hereafter abbreviated as VIP) struggle with navigating independently as well as perceiving their environment. For them, scene understanding is not just another convenience, but an important requirement to aid in their independence and safety [2].

Traditionally, assistive technologies have relied on computer vision and deep learning models. These models help in detecting separate objects or avoiding obstacles, such as the You-Only-Look-Once (YOLO) models. However, these models lack in providing semantic relationships between objects and spatial context, which is essential for providing a “close-to-human” experience [3].

More recently, Vision-Language Models (VLMs) have significantly revolutionized this domain. VLMs emerged with the integration of vision encoders with the Large-Language Models (LLMs), such as CLIP, essentially giving “eyes” to the LLMs. VLMs are now able to provide an intuitive as well

as more detailed description of the scene that early assistive technologies lacked [4].

Despite all the capabilities of VLMs, one bottleneck remains: deployment. State-of-the-Art VLMs like GPT-4V feature billions of parameters that require an immense amount of computational resources for processing and have huge power requirements [5]. Consequently, they are usually cloud-based services relying on massive data centers. However, this introduces privacy concerns and internet-dependence for these assistive devices. To counter this, numerous research is being conducted on deploying VLMs on edge devices through different techniques such as model quantization, pruning and knowledge distillation [6]. For the purpose of this research, we will be focusing on:

- Deploying quantized VLMs on NVIDIA’s Jetson Orin Nano — an edge device.
- Measuring the performance of five local VLMs on multiple hardware benchmarks such as time-to-first-token (TTFT), throughput (token/s), power consumption, and memory (RAM and Swap) usage.
- Assessing model precision benchmarks such as CLIPScore for image-text pair compatibility and Polling based Object Probing Evaluation (POPE) for object hallucination detection.

II. RELATED WORK

A. Edge Deployment and Hardware Benchmarking

VLMs are capable of performing difficult tasks such as image captioning and visual question answering, which are essential for assistive devices. However, the deployment of VLMs on the edge remains a challenge due to their memory and power limitations, which has been extensively surveyed by Sharshar *et al.* [7]. To manage these constraints, hardware benchmarking is crucial. For instance, Shakya *et al.* have benchmarked object detection models such as YOLO variants

on an aerial dataset, evaluating these models on latency, memory usage, and power consumption on NVIDIA's Jetson Orin Nano (8GB) [8]. Similarly, Park *et al.* point out the need for testing open-vocabulary models such as YOLO-World on their accuracy and latency for mobile robot applications on NVIDIA's Jetson Orin AGX (64GB) — a high end edge device [9]. These studies emphasize benchmarking on edge hardware; however, they evaluate object-detection models rather than VLMs specifically.

B. Existing Frameworks for the Visually Impaired

As noted earlier, VLMs demonstrate a huge potential in environmental and situational awareness for the visually impaired. In light of this, Mahboob *et al.* developed an agentic framework capable of scene understanding for the visually impaired; however, they rely on cloud-based VLMs for their processing which makes their system network dependent [10]. In contrast, local VLMs promise low-latency, on edge deployment, and privacy-preserving systems. Bappy *et al.* utilized lightweight AI models to develop a complete framework for the visually impaired. They used SSD-Mobile-NET for real time object detection and VILA-1.5-3b for scene comprehension, both deployed on Jetson Orin Nano. However, their paper lacks evaluation of VLMs on precision, hallucinations, and hardware-specific benchmarking such as memory and power consumption [11].

Gurari *et al.* have talked about the importance of using datasets containing images and questions created by blind or visually impaired (hereafter abbreviated as VI) individuals for evaluating frameworks of assistive devices for their use, subsequently proposing the first Visual Question Answering (VQA) dataset containing images and questions from VI individuals [12].

Other works in developing assistive technologies either rely on more powerful edge hardware or compromise on using datasets that are not specific to VI individuals. For instance, Baghel *et al.* deployed and tested two variants of SmolVLM, a local VLM, on AVCaps and Charades datasets [13]. However, these datasets are not specific to VI use-cases and contain the same limitations discussed in VizWiz [12]. Moreover, Kwon *et al.* deployed a VLM-TTS system for a robotic guide dog based on larger variants of local VLMs such as LLaVA-1.5-7B, Qwen2-VL-7B, and Qwen2.5-VL-7B. They benchmarked hardware metrics such as time-to-first-token (TTFT) and throughput, however, their implementation relies on NVIDIA's Jetson Orin AGX (64GB) [14].

C. Evaluation of VLMs and Hallucination Problems

VLMs differ significantly in their architecture from object detection and classification models such as YOLO. While YOLO relies on Convolutional Neural Network (CNN)-based backend [15], VLMs rely on transformer-based architecture that combine images with text to reason [16]. Hence evaluating them on reference based image captioning metrics such as BLEU-1, SPICE, METEOR, CIDER-D provide inaccurate results that don't capture the full scope of the output often

penalizing the use of synonyms generated as an alternative to the ground truth captions as noted in CLIPScore. [17].

Taking this into account, Hani *et al.* have comprehensively evaluated VLMs on CLIPScore as well as computed results by having human-as-a-judge measuring fluency of VLMs. However, they have analyzed only larger parameter models such as LLaVA-13B on general datasets such as Microsoft Common Objects in Context (MSCOCO) [18].

It is important to note that standard VLMs benchmarking fail to consider model hallucinations, even after utilizing CLIPScore. For VI individuals, a hallucinated object (i.e. the VLM imagining a clear path when there are dangerous obstacles in the way) poses a safety risk. Hence, it is important to incorporate benchmarking of hallucinations as proposed by Li *et al.* for measuring model feasibility in assistive devices [19].

In summary, the existing literature highlights the potential of using VLMs for VI individuals, however, current solutions either rely on cloud-based VLMs, use high-end edge devices, or use datasets not tailored for the VIP. Furthermore, studies focusing on edge deployment either evaluate standard-object detection models or lack hardware-specific and hallucination benchmarking. To address these gaps, we evaluate five local and open-source models on a resource-constrained edge device (NVIDIA's Jetson Orin Nano). These models are tested on the VizWiz-VQA dataset that contains images captured by the VI individuals from their smartphones. We present a comprehensive analysis of hardware metrics (latency, throughput, power, memory usage) as well as measuring semantic reliability using CLIPScore and object hallucinations using the POPE benchmark.

III. METHODOLOGY

A. Hardware Setup

All experiments are conducted with NVIDIA's Jetson Orin Nano Developer Kit. In Jetson Orin Nano both the 6-core ARM Cortex-A78AE CPU as well as 1024-core NVIDIA's Ampere GPU access the same 128-bit LPDDR5 8GB of RAM [20]. For most VLMs, even the quantized ones, this is not enough memory for inference. To prevent out-of-memory (OOM) errors we have allocated a virtual memory of 100GB on the NVMe SSD attached externally to the device, moreover, we have enabled MAXN SUPER Mode enabling the device to consume unconstrained power during inference.

B. Software Setup

1) *OS Configuration:* The Jetson Orin Nano is configured to run Jetpack 6.2, the latest version of Jetpack 6. It supports Super Mode capable of delivering high power on the Jetson Orin series. With its updates, it promises to deliver upto 2 times higher generative AI inference performance than before [21].

2) *Deployment frameworks for VLMs:* There are several methods to deploy VLMs. However, as noted by Kwon *et al.* only two are significant for the Jetson Orin series — **Hugging Face Transformers** and **llama.cpp** backend. This is due to

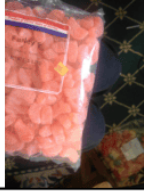
		
Prompt	What's the name of this product?	What color do these look?
Ground Truth	basil leaves	orange/pink
gemma3:4b	Based on the image, the product is "Value Brand Basil Leaves" .	Based on the image, the slices appear to be a vibrant "orange" color. They are a bright, saturated orange, as indicated by the "Orange Slices" label on the package.
gemma3:12b	Based on the image, the product name is "Basil Leaves" .	The candies in the bag appear to be "orange" . They have a bright, slightly pinkish-orange hue.
ministral-3:8b	The product in the image is labeled as "Basil Leaves" . The container holds dried basil leaves.	The orange slices in the image appear to be a bright, vibrant "orange-red" color. They look fresh and juicy, typical of the color of many commercially packaged orange slices.
qwen3-vl:2b-instruct	Based on the image provided, the product name is "Basil Leaves" . This is clearly visible on the label of the spice container. The label also indicates that the net weight is 0.62 oz (17g).	Based on the image provided, the candies look orange. However, there are a few important details to note about the color: The "color" of the candy is "orange" . The "color of the packaging" is "white" , as the label on the bag is printed in white. In this case, the color of the candy is the dominant color, so the answer is "orange" .
qwen3-vl:4b-instruct	Based on the image provided, the product is "Basil Leaves" . This is indicated by the prominent green label on the jar, which clearly says "Basil Leaves". The jar appears to contain dried basil leaves, and the net weight is listed as 0.62 oz (17g). The brand name "Value" is partially visible at the top of the label. Therefore, the name of this product is "Basil Leaves" .	The candy in the bag looks "orange" . The label on the bag says "Farley's Orange Slices" and the candies themselves have a bright orange color, which is typical for orange-flavored hard candies or candy slices. The plastic bag is clear, so the color is clearly visible. A vibrant, sugary orange. (Note: The price tag shows \$6.55, which seems unusually high for a 5-pound bag of candy, but that's a separate detail.)

Fig. 1. Model-specific VLM outputs compared to ground truth descriptions for random sample images

their ARM compatibility and ease of integration [14]. In our experiments llama.cpp is selected because it utilizes both the CPU and GPU during inference [22]. Moreover, llama.cpp requires the model to be converted to GPT-Generated Unified Format (GGUF) which compresses the model weights using "K-Quantization" Techniques [23]. Our experiments will be conducted through Ollama which is a wrapper for llama.cpp [24].

C. Dataset Selection

The evaluation of VLM on standard datasets such as the MSCOCO dataset does not capture the lived experiences of VI individuals [25]. Our goal is to select a dataset that contains real-world imperfections. Hence, in our benchmarking, we will be evaluating the VizWiz dataset — a VQA dataset collected by VI individuals through their smartphones. This dataset contains images with motion blur, poor lighting and poor framing, making it ideal for our use-case. We will randomly select 50 images from the validation dataset available on their website for our benchmarking [12]. Fig. 2 shows images from the VizWiz dataset.

D. Benchmarking Procedure

The image-question pairs within the dataset are used to prompt the selected VLMs: gemma3:4b [26], gemma3:12b [27], ministral-3:8b [28], qwen3-vl:4b-instruct [29], qwen3-vl:4b-instruct [30]. The associated prompts, ground truth, and the generated VLM responses on the dataset images are shown in Fig. 1.

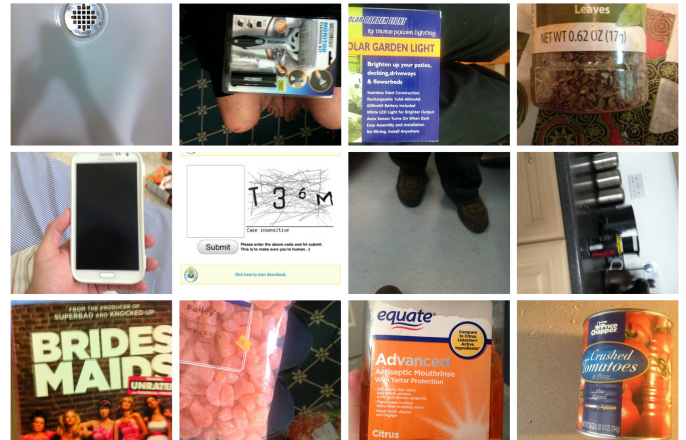


Fig. 2. Images from the VizWiz dataset

IV. EVALUATION METRICS

A. Hardware-Specific Benchmarks

We measure **latency**, **throughput**, **power consumption** and **memory usage** for each VLM. Latency is the time taken by the VLM to generate the first token, also called time-to-first-token and throughput is the amount of tokens it generates per second. Power consumption is monitored using `tegrastats` which is available on Jetson devices. Finally, memory utilization is measured across both main memory and swap space.

B. Accuracy and Hallucination

1) **CLIPScore**: CLIPScore is calculated by taking the raw cosine similarity between the VLM generated caption and the

image, as shown in (1).

$$\text{CLIPScore}(I, c) = \max \left(0, w \times \frac{\mathbf{v}_I \cdot \mathbf{v}_c}{\|\mathbf{v}_I\| \|\mathbf{v}_c\|} \right) \quad (1)$$

Where I is the input image, c is the generated caption, and $\mathbf{v}_I$ and $\mathbf{v}_c$ denote their respective CLIP embeddings. The weight parameter w is typically set to 2.5.

It determines the accuracy of VLM generated output without relying on exact matches of words which is traditionally used in image captioning evaluation. It is a reference-free metric emphasizing on image-text compatibility [17].

2) *Polling-based Object Probing Evaluation (POPE)*: VLMs are prone to hallucinating objects in an image. To evaluate this tendency, POPE queries the VLMs with Yes-or-No questions about the existence of different objects that may or may not be present in the image. This benchmark is critical in assistive devices for the visually impaired.

We determined POPE by prompting the VLMs with 4 Yes-or-No questions per image. One question queries about the ground truth, and the other three queries random objects not present in the image.

V. RESULTS & DISCUSSION

A. Latency & Throughput

As shown in Fig. 3 and Table I the latency and the throughput of the models primarily depend on the model sizes. For instance, models exceeding 4 billion parameters demonstrated high Time to First Token (TTFT). gemma3:12b averaged at 17.7 s while ministral-3:8b averaged at 27.9 s. The smaller models outperformed the larger parameter models, however our tests demonstrate a notable variance in stability of TTFT. For instance, qwen3-vl:2b-instruct and qwen3-vl:4b-instruct showed the fastest minimum latency of 0.21 s and 0.41 s respectively, however, they suffered with the highest variance with a maximum of 6.2 s and 8.36 s respectively. This variance is likely due to the diversity in the VizWiz dataset as well as the Dynamic Resolution architecture of Qwen models that let them assign different number of visual tokens to different resolution of images. The gemma3:4b model demonstrated the lowest as well as the most stable TTFT averaging at 0.5 s within a short range of 0.46 s to 0.66 s.

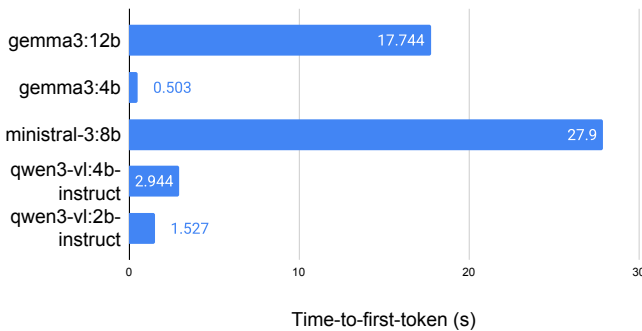


Fig. 3. Average TTFT of evaluated models

In terms of throughput, qwen3-vl:2b-instruct outperformed all other models with an average of 32.7 tokens/s as shown in Fig. 4, largely attributable to its relatively smaller model size. ministral-3:8b and gemma3:12b demonstrated a considerably low response rate of 0.52 and 0.128 tokens/s respectively. Due to their high latency and low throughput, these models are unsuitable for assistive technologies.

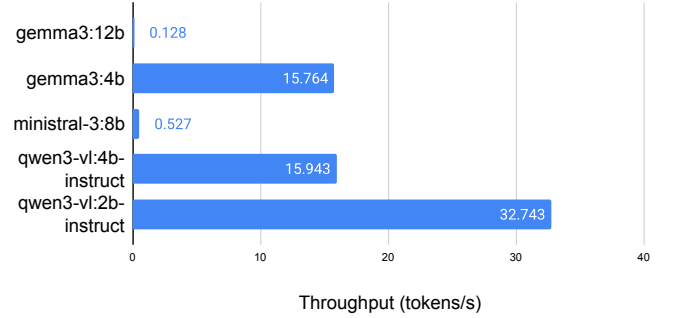


Fig. 4. Average Tokens per Second of evaluated models

B. Memory

The primary cause for low inference speeds of larger models is the memory limit of the Jetson Orin Nano (8GB). While smaller models consumed less than 2 GB of virtual memory, the larger models require an immense amount of memory as illustrated in Fig. 5 and 6. For instance, ministral-3:8b and gemma3:12b consumed an average of 8.06 and 7.23 GB of virtual memory respectively. These models being heavily reliant on virtual memory face a severe memory bottleneck, as the system spends most of its time in moving model weights from the SSD to the GPU for inference.

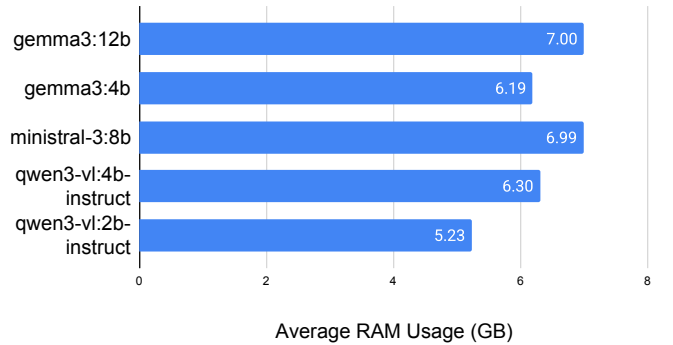


Fig. 5. Average RAM Usage of evaluated models

C. Power Consumption

Contrary to what was expected, the smaller and faster models consumed significantly more power during inference compared to the larger models, as shown in Fig. 7. This is due to the higher GPU utilization of smaller models. The GPU being the most power intensive unit of the Jetson, remains

TABLE I
HARDWARE METRICS COMPARISON ACROSS MODELS

Model	Latency (s)		Throughput (tokens/s)		Memory (GB)		Power (W)	GPU (%)
	Min	Max	Min	Max	RAM	Swap		
gemma3:12b	17.418	18.034	0.12	0.14	6.99	7.22	7.851	5.255
gemma3:4b	0.463	0.664	13.78	17.13	6.19	1.36	17.824	93.927
ministral-3:8b	16.195	39.207	0.09	0.527	6.99	8.06	8.106	10.845
qwen3-vl:4b-instruct	0.413	8.366	15.52	16.54	6.30	1.99	19.485	94.955
qwen3-vl:2b-instruct	0.21	6.208	30.76	35.96	5.22	1.36	18.505	85.2

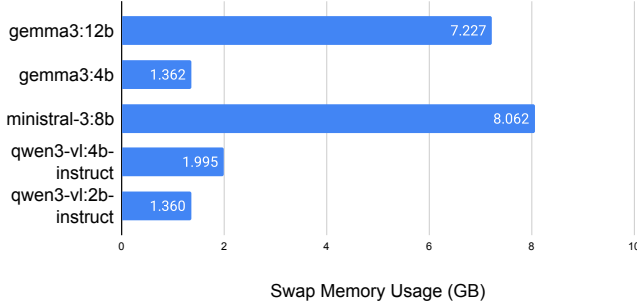


Fig. 6. Average Swap Usage of evaluated models

largely idle for the larger models as the data transfer between the virtual memory and the GPU is significantly slower.

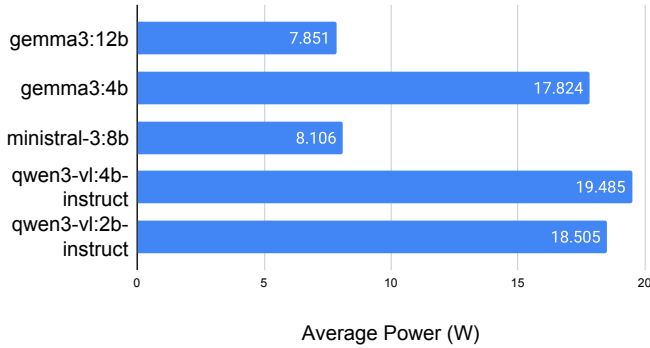


Fig. 7. Average Power Consumption of evaluated models

D. CLIPScore

The CLIPScores of the local models are compared with a massive trillion parameter model like Gemini. The average scores of the models are nearly identical. This indicates an information bottleneck attributable to the noisy, blurry, and poorly framed images found in the VizWiz dataset. Consequently, the larger models can not extract more features than the smaller models. Table II shows the average, minimum and maximum scores across different models.

E. POPE

Table III compares the POPE scores across various models. In the POPE metric, high recall means that the model finds every object present in the image, and high precision indicates

TABLE II
CLIPSCORE

Model	Average	Min	Max
Gemini 3	0.30	0.25	0.35
gemma3:12b	0.29	0.21	0.35
gemma3:4b	0.29	0.21	0.35
ministral-3:8b	0.28	0.23	0.33
qwen3-vl:4b-instruct	0.29	0.19	0.35
qwen3-vl:2b-instruct	0.29	0.21	0.36

that the model mostly does not hallucinate objects. Taking this into account, gemma3:4b demonstrates the most hallucinations (Low Precision: 0.6471) but correctly detects the present objects (High Recall: 1.0000). Meanwhile, ministral-3:8b shows the least amount of object hallucinations (High Precision: 1.0000) but missed half the objects in the images (Low Recall: 0.5455). Moreover, the 12b variant of gemma3 demonstrated better results than its 4b counterpart in precision and F1-Score. The most notable performance was of qwen3-vl:2b-instruct demonstrating the highest accuracy as well as F1-score despite being the smallest of the selected models.

TABLE III
POPE

Model	Accuracy	Precision	Recall	F1 Score
gemma3:4b	0.86	0.65	1.00	0.78
gemma3:12b	0.95	0.91	0.91	0.90
ministral-3:8b	0.89	1.00	0.54	0.71
qwen3-vl:2b-instruct	0.95	0.85	1.00	0.92
qwen3-vl:4b-instruct	0.93	0.90	0.82	0.86

VI. CONCLUSION

In this paper, five local, open-source VLMs were deployed and evaluated on hardware and accuracy benchmarks on NVIDIA's Jetson Orin Nano (8GB), exploring their feasibility in privacy-preserving and offline scene description applications for the VIP. To ensure practicality of these models, VizWiz-VQA dataset was utilized in our benchmarking which consists of blurry, poorly framed and noisy images taken by the VIP. However, the memory constraint of 8 gigabytes in NVIDIA's Jetson Orin Nano served as a major bottleneck in inference of models. For instance, larger models that required high memory usage demonstrated high latencies (TTFT) as well as low throughput. Contrary to what was expected, larger models consumed less power during inference as the GPU remained mostly idle waiting for data transfer from the swap space to

the GPU. Additionally, semantic evaluations using CLIPScore revealed that local VLMs demonstrate comparable scores to cloud-based models such as Gemini. This demonstrates that using bigger models for visually impaired applications does not yield better scene description as the images taken by the VIP contain limited usable information. In terms of object hallucinations as determined by the POPE metric, smaller and faster models were more prone to output hallucinatory responses than the larger models, indicating a critical trade-off between user safety and low overall latency. To conclude, small local VLMs on edge devices have a good prospect of being a low latency and private alternative to cloud based VLMs, however, future work must focus on mitigating object hallucinations demonstrated by the models to assure the safety of the VIP.

REFERENCES

- [1] "Vision impairment and blindness." [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment>
- [2] G. N. Alwakid, M. Humayun, and Z. Ahmad, "Computer-Vision and Edge-Enabled Real-Time Assistance Framework for Visually Impaired Persons with LPWAN Emergency Signaling," *Sensors (Basel, Switzerland)*, vol. 25, no. 22, p. 7016, Nov. 2025. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12655979/>
- [3] L. Liu, A. Kudaeva, M. Cipriano, F. A. Ghannam, F. Tan, G. de Melo, and A. Sevsuk, "Mingle: VLMs for semantically complex region detection in urban scenes," 2026. [Online]. Available: <https://arxiv.org/abs/2509.13484>
- [4] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," 2021. [Online]. Available: <https://arxiv.org/abs/2103.00020>
- [5] Y. Yao, T. Yu, A. Zhang, C. Wang, J. Cui, H. Zhu, T. Cai, C. Chen, H. Li, W. Zhao, Z. He, Q. Chen, R. Zhou, Z. Zou, H. Zhang, S. Hu, Z. Zheng, J. Zhou, J. Cai, X. Han, G. Zeng, D. Li, Z. Liu, and M. Sun, "Efficient GPT-4V level multimodal large language model for deployment on edge devices," *Nature Communications*, vol. 16, no. 1, p. 5509, Jul. 2025. [Online]. Available: <https://doi.org/10.1038/s41467-025-61040-5>
- [6] F. Piccialli, D. Chiaro, P. Qi, V. Bellandi, and E. Damiani, "Federated and edge learning for large language models," *Information Fusion*, vol. 117, p. 102840, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1566253524006183>
- [7] A. Sharshar, L. U. Khan, W. Ullah, and M. Guizani, "Vision-language models for edge networks: A comprehensive survey," *IEEE Internet of Things Journal*, vol. 12, no. 16, pp. 32 701–32 724, 2025.
- [8] B. Shakya, O. El-Gayar, J. Kaabi, and K. Mamun Ahmed, "Small-object detection at the edge: A pareto-efficient benchmark of lightweight yolo models on uav and overhead datasets," *IEEE Access*, vol. 14, pp. 528–548, 2026.
- [9] J. Park, P. Kim, and D. Ko, "Real-time open-vocabulary perception for mobile robots on edge devices: a systematic analysis of the accuracy-latency trade-off," *Frontiers in Robotics and AI*, vol. Volume 12 - 2025, 2025. [Online]. Available: <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2025.1693988>
- [10] Z. Mahboob, S. Sadiq, U. S. Khan, A. Zeb, and M. I. Tiwana, "Masa: Multi agent situational awareness for visually impaired," in *2024 21st International Bhurban Conference on Applied Sciences and Technology (IBCAST)*, 2024, pp. 349–354.
- [11] A. S. Bappy, T. Seppänen, and M. Z. Hoque, "SENSEYE: A Resource-Aware Visionary Framework for Assisting Individuals with Visual Disabilities," Nov. 2025, iSSN: 2693-5015. [Online]. Available: <https://www.researchsquare.com/article/rs-8117793/v1>
- [12] D. Gurari, Q. Li, A. J. Stangl, A. Guo, C. Lin, K. Grauman, J. Luo, and J. P. Bigham, "Vizwiz grand challenge: Answering visual questions from blind people," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.
- [13] S. S. Baghel, Y. P. S. Rathore, A. Pradhan, S. Jena, A. Bhavsar, A. Shukla, and P. Goyal, "Towards blind and low-vision accessibility of lightweight VLMs and custom LLM-evals," in *Proceedings of the 1st Workshop on Multimodal Models for Low-Resource Contexts and Social Impact (MMLoSo 2025)*, A. Shukla, S. Kumar, A. S. Bedi, and T. Chakraborty, Eds. Mumbai, India: Association for Computational Linguistics, Dec. 2025, pp. 86–94. [Online]. Available: <https://aclanthology.org/2025.mmloso-1.8/>
- [14] J. Kwon, J. Lee, and Y. Kwon, "Design practices and lessons from deploying on-device vision-language interaction in robotic guide dogs," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, October 2025, pp. 2435–2444.
- [15] V. V. C. R. K. and R. A. C., "Real time object detection system with yolo and cnn models: A review," 2022. [Online]. Available: <https://arxiv.org/abs/2208.00773>
- [16] F. Bordes, R. Y. Pang, A. Ajay, A. C. Li, A. Bardes, S. Petryk, O. Mañas, Z. Lin, A. Mahmoud, B. Jayaraman *et al.*, "An introduction to vision-language modeling," *arXiv preprint arXiv:2405.17247*, 2024.
- [17] J. Hessel, A. Holtzman, M. Forbes, R. Le Bras, and Y. Choi, "CLIPScore: A reference-free evaluation metric for image captioning," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 7514–7528. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.595/>
- [18] A. Hani, N. Tagougui, and M. Kherallah, "Toward human-level understanding: A systematic review of vision-language models for image captioning," *International Arab Journal of Information Technology (IAJIT)*, vol. 23, no. 1, 2026.
- [19] Y. Li, Y. Du, K. Zhou, J. Wang, W. X. Zhao, and J.-R. Wen, "Evaluating object hallucination in large vision-language models," 2023. [Online]. Available: <https://arxiv.org/abs/2305.10355>
- [20] "Jetson Orin Nano Series Modules Data Sheet." [Online]. Available: <https://nvdam.widen.net/s/zkfjgimtds2/jetson-orin-datasheet-nano-developer-kit-3575392-r2>
- [21] "JetPack SDK." [Online]. Available: <https://developer.nvidia.com/embedded/jetpack-sdk-62>
- [22] "ggml-org/llama.cpp," Feb. 2026, original-date: 2023-03-10T18:58:00Z. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [23] U. Kurt, "Which Quantization Should I Use? A Unified Evaluation of llama.cpp Quantization on Llama-3.1-8B-Instruct," Jan. 2026, arXiv:2601.14277 [cs]. [Online]. Available: <http://arxiv.org/abs/2601.14277>
- [24] "Ollama." [Online]. Available: <https://ollama.com>
- [25] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *Computer Vision – ECCV 2014*, D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, Eds. Cham: Springer International Publishing, 2014, pp. 740–755.
- [26] "google/gemma-3-4b-it · Hugging Face," Jan. 2026. [Online]. Available: <https://huggingface.co/google/gemma-3-4b-it>
- [27] "google/gemma-3-12b-it · Hugging Face," Jan. 2026. [Online]. Available: <https://huggingface.co/google/gemma-3-12b-it>
- [28] "mistralai/Mistral-3-8B-Instruct-2512-GGUF · Hugging Face," Dec. 2025. [Online]. Available: <https://huggingface.co/mistralai/Mistral-3-8B-Instruct-2512-GGUF>
- [29] "Qwen/Qwen3-VL-4B-Instruct · Hugging Face." [Online]. Available: <https://huggingface.co/Qwen/Qwen3-VL-4B-Instruct>
- [30] "Qwen/Qwen3-VL-2B-Instruct · Hugging Face." [Online]. Available: <https://huggingface.co/Qwen/Qwen3-VL-2B-Instruct>